

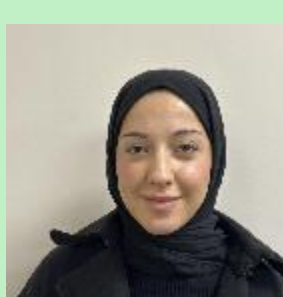
# SP-201 RED

## Multi-Agent Research & Fact-Checking Pipeline

CS4850 | Section 2 | Spring 2026

April 12, 2026

Professor Perry

|                                                                                    |                                                                                    |                                                                                     |                                                                                      |
|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
|  |  |  |  |
| Melike Ozcelik<br>Team Leader<br>Developer                                         | Raaid Khan<br>Developer<br>Documentation                                           | Nha Phan<br>Developer                                                               | Afra Kurudirek<br>Documentation                                                      |

### Revision History

| Date              | Version | Author                | Description                                                                                 |
|-------------------|---------|-----------------------|---------------------------------------------------------------------------------------------|
| February 28, 2026 | 0.1     | Afra Kurudirek        | Initial draft of Software Test Plan (STP) structure created using course template.          |
| March 15, 2026    | 0.2     | Melike Ozcelik        | Added testing scope, objectives, and overall test strategy sections.                        |
| March 29, 2026    | 0.3     | Nha Phan & Raaid Khan | Updated integration and system testing details; aligned test cases with SRS requirements.   |
| April 8, 2026     | 0.4     | Afra Kurudirek        | Completed Sections 7–11 including environment, milestones, training, and defect management. |

# Table of Contents

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>1. Test Objectives and Scope .....</b>                   | <b>5</b>  |
| <b>1.1 Test Objectives.....</b>                             | <b>5</b>  |
| <b>1.2 Scope of Testing .....</b>                           | <b>5</b>  |
| 1.2.1 In-Scope Testing.....                                 | 5         |
| 1.2.2 Out-of-Scope Testing.....                             | 6         |
| <b>2. Testing Summary .....</b>                             | <b>6</b>  |
| <b>2.1 Scope of Testing .....</b>                           | <b>6</b>  |
| 2.1.1 In Scope .....                                        | 6         |
| 2.1.2 Out of Scope.....                                     | 7         |
| <b>2.2 Test Coverage Summary .....</b>                      | <b>7</b>  |
| <b>3. Analysis of Scope and Test Focus Areas .....</b>      | <b>8</b>  |
| <b>3.1 Release Content.....</b>                             | <b>8</b>  |
| <b>3.2 Key Test Focus Areas .....</b>                       | <b>8</b>  |
| <b>3.3 Regression Testing .....</b>                         | <b>8</b>  |
| <b>3.4 Platform and Environment Testing .....</b>           | <b>9</b>  |
| <b>4. Progression Test Objectives .....</b>                 | <b>10</b> |
| <b>4.1 Progression Testing Approach.....</b>                | <b>10</b> |
| <b>5. Regression Test Objectives .....</b>                  | <b>11</b> |
| <b>5.1 Regression Testing Goals.....</b>                    | <b>11</b> |
| <b>5.2 Regression Testing Approach.....</b>                 | <b>13</b> |
| <b>6. Other Testing.....</b>                                | <b>13</b> |
| <b>6.1 Security Testing.....</b>                            | <b>13</b> |
| <b>6.2 Stress &amp; Volume Testing (Load Testing) .....</b> | <b>14</b> |
| <b>6.3 Connectivity Testing .....</b>                       | <b>15</b> |
| <b>6.4 Disaster Recovery / Backup Testing .....</b>         | <b>15</b> |
| <b>6.5 Unit Testing .....</b>                               | <b>16</b> |
| <b>6.6 Integration Testing.....</b>                         | <b>16</b> |
| <b>7. Test Strategy .....</b>                               | <b>17</b> |
| <b>7.1 Test Level Responsibilities .....</b>                | <b>17</b> |
| <b>7.2 Test Types &amp; Approach.....</b>                   | <b>18</b> |
| 7.2.1 Functional (Progression) Testing.....                 | 18        |
| 7.2.2 Regression Testing.....                               | 18        |
| 7.2.3 Performance & Load Testing .....                      | 18        |

|                                                             |           |
|-------------------------------------------------------------|-----------|
| 7.2.4 Security Testing .....                                | 18        |
| <b>7.3 Build Strategy .....</b>                             | <b>19</b> |
| <b>7.4 Test Execution Schedule.....</b>                     | <b>19</b> |
| <b>7.5 Facility, Data, and Resource Provision Plan.....</b> | <b>20</b> |
| 7.5.1 Test Environments .....                               | 20        |
| 7.5.2 Access to External Systems .....                      | 21        |
| 7.5.3 Testing Requirements .....                            | 21        |
| 7.5.4 Data Requirements.....                                | 22        |
| 7.5.5 Resources & Skills .....                              | 22        |
| <b>7.6 Testing Tools .....</b>                              | <b>22</b> |
| <b>7.7 Testing Handover Procedure.....</b>                  | <b>23</b> |
| <b>7.8 Testing Metrics .....</b>                            | <b>23</b> |
| <b>8. Test Environment Plan .....</b>                       | <b>24</b> |
| 8.1 Test Environment Map .....                              | 24        |
| 8.2 Test Environment Details.....                           | 24        |
| 8.2.1 Testers .....                                         | 24        |
| 8.2.2 Hardware and Firmware .....                           | 25        |
| 8.2.3 Software .....                                        | 25        |
| 8.2.4 Interfaces .....                                      | 26        |
| 8.2.5 Other Materials.....                                  | 26        |
| 8.3 Establishing Environment .....                          | 26        |
| 8.4 Environment Control .....                               | 27        |
| 8.5 Environment Roles & Responsibilities.....               | 27        |
| <b>9. Assumptions and Dependencies .....</b>                | <b>27</b> |
| 9.1 Assumptions .....                                       | 27        |
| 9.2 Dependencies .....                                      | 28        |
| <b>10. Entry and Exit Criteria .....</b>                    | <b>29</b> |
| 10.1 Entry Criteria .....                                   | 29        |
| 10.1.1 Unit Testing Entry Criteria .....                    | 29        |
| 10.1.2 Integration Testing Entry Criteria .....             | 29        |
| 10.1.3 System Testing Entry Criteria .....                  | 29        |
| 10.1.4 User Acceptance Testing (UAT) Entry Criteria.....    | 29        |
| 10.2 Exit Criteria .....                                    | 30        |
| 10.2.1 Unit Testing Exit Criteria.....                      | 30        |
| 10.2.2 Integration Testing Exit Criteria .....              | 30        |
| 10.2.3 System Testing Exit Criteria.....                    | 30        |
| 10.2.4 User Acceptance Testing Exit Criteria .....          | 30        |
| 10.3 Final Release Exit Criteria .....                      | 30        |
| <b>11. Administrative Plan .....</b>                        | <b>31</b> |
| 11.1 Approvals .....                                        | 31        |

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>11.2 Test Milestones and Schedule.....</b>      | <b>31</b> |
| <b>11.3 Training .....</b>                         | <b>32</b> |
| <b>11.4 Defect Management .....</b>                | <b>32</b> |
| 11.4.1 Defect Tracking Tool .....                  | 32        |
| <b>11.5 Severity Classification .....</b>          | <b>33</b> |
| 11.5.1 Defect Resolution Process .....             | 33        |
| 11.5.2 Regression Testing.....                     | 33        |
| <b>12. Definitions .....</b>                       | <b>34</b> |
| <b>13. References.....</b>                         | <b>35</b> |
| <b>14. Points of Contact.....</b>                  | <b>35</b> |
| <b>15. Software Test Report (STR) .....</b>        | <b>36</b> |
| 15.1 Test Execution Summary.....                   | 36        |
| 15.2 Progression (Functional) Test Results .....   | 37        |
| 15.3 Regression Test Results .....                 | 38        |
| 15.4 Performance & Load Testing Results .....      | 39        |
| 15.5 Security & Connectivity Testing Results ..... | 39        |
| 15.6 Defect Summary .....                          | 39        |
| 15.7 Final Testing Conclusion .....                | 40        |

# 1. Test Objectives and Scope

## 1.1 Test Objectives

The objective of this Software Test Plan (STP) for SP-201 Red: Agentic Multi-Agent Fact-Verification System is to verify and validate that the system functions according to the requirements defined in the Software Requirements Specification (SRS) and aligns with the architecture described in the Software Design Document (SDD).

Specifically, this testing effort aims to:

- Verify that the Manager Agent correctly orchestrates multi-agent workflow.
- Validate that the Search Agent retrieves relevant external information via API calls.
- Ensure that the Synthesizer Agent generates structured draft responses.
- Confirm that the Fact-Checker Agent validates claims accurately.
- Verify correct session state transitions throughout execution.
- Validate end-to-end multi-agent coordination.
- Ensure stable API connectivity and proper error handling.
- Identify, document, and resolve defects prior to final submission.

Testing ensures that SP-201 Red performs reliably, securely, and according to project specifications.

## 1.2 Scope of Testing

### 1.2.1 In-Scope Testing

The following system components and features are included in testing:

- Manager Agent orchestration logic
- Search Agent external data retrieval
- Synthesizer Agent draft generation
- Fact-Checker validation workflow
- Session state management
- End-to-end multi-agent coordination
- API connectivity and response handling
- Functional validation against SRS requirements

Testing includes:

- Unit Testing
- Integration Testing

- Functional Validation
- API Connectivity Testing
- Regression Testing

### 1.2.2 Out-of-Scope Testing

The following areas are not included in this testing phase:

- Third-party API internal logic
- External service availability guarantees
- Long-term production scalability testing
- Formal security penetration testing
- Mobile platform optimization

This Software Test Plan defines how SP-201 Red will be validated and provides the framework for documenting results in the Software Test Report (STR) section.

## 2. Testing Summary

### 2.1 Scope of Testing

This section summarizes the overall testing coverage for the SP-201 Red Agentic Multi-Agent Fact-Verification System.

Testing focuses on verifying that all core system components function according to the SRS and integrate correctly within the multi-agent pipeline.

#### 2.1.1 In Scope

The following system components and behaviors are included in testing:

- Prompt decomposition by the Manager Agent
- External search API integration
- Citation extraction and preservation
- Markdown output formatting
- Fact-check validation loops

- Workflow state transitions:
  - Pending → Decomposing → Searching → Synthesizing → Fact-Checking → Finalizing
- Retry logic and error handling mechanisms
- End-to-end multi-agent coordination

The following types of testing will be performed:

- Unit Testing of individual agents
- Integration Testing between agents
- Functional Testing against defined requirements
- API Connectivity Testing
- Regression Testing after defect resolution

### 2.1.2 Out of Scope

The following areas are excluded from testing in this phase:

- Enterprise-level scalability testing
- Multi-user concurrency stress testing
- Paid or premium API integrations
- Long-term persistent memory validation
- Production deployment validation
- Formal penetration/security testing

## 2.2 Test Coverage Summary

Testing is designed to ensure:

- All defined functional requirements are validated
- Agent interactions occur in the correct sequence
- Error conditions are handled gracefully
- API failures do not crash the system
- Output formatting meets specification requirements

The Software Test Report (STR) section will document Pass/Fail results and defect severity for each test case.

## 3. Analysis of Scope and Test Focus Areas

### 3.1 Release Content

The current release of SP-201 Red includes the following implemented components:

- Fully implemented Manager–Worker multi-agent architecture
- Search Agent integration using DuckDuckGo API
- Synthesizer Agent integration using Gemini API
- Fact-Checker Agent validation workflow
- Session State Manager for workflow tracking
- Command-Line Interface (CLI) interaction layer

This release represents a functional prototype capable of executing the complete multi-agent fact-verification pipeline.

### 3.2 Key Test Focus Areas

Testing efforts will prioritize verification of the following critical system behaviors:

- Accurate prompt decomposition by the Manager Agent
- Correct API request/response handling for external search
- Proper citation extraction and formatting
- Deterministic workflow state transitions
- Stability of retry logic and exception handling
- Correct interaction between agents in end-to-end execution

Special attention will be given to integration points between agents, as these represent the highest risk areas for failure.

### 3.3 Regression Testing

Regression testing will be performed after:

- Modifications to agent logic
- Updates to API integration
- Bug fixes impacting workflow execution
- Changes to state transition handling



Regression testing will verify that:

- Previously functioning features remain operational
- Citation formatting remains consistent
- Workflow transitions remain stable
- Retry and error-handling mechanisms do not introduce new defects

### 3.4 Platform and Environment Testing

Testing will be conducted under the following environments:

- macOS
- Windows 11
- Python 3.10+ runtime
- Google ADK framework
- Gemini Free Tier API access
- Internet-enabled testing environment

Testing across these environments ensures compatibility and functional consistency.

## 4. Progression Test Objectives

This section defines the high-level progression test objectives for SP-201 Red. These objectives verify that core system functionality operates according to the Software Requirements Specification (SRS) and Software Design Document (SDD).

Each objective supports requirements traceability and will be validated through defined test cases documented in later sections.

| Ref    | Function              | Test Objective                       | Evaluation Criteria                    | X-Ref             | P      |
|--------|-----------------------|--------------------------------------|----------------------------------------|-------------------|--------|
| TC-01  | Prompt Decomposition  | Verify prompt is split into $\geq 3$ | Sub-questions generated and structured | SRS 3.1           | High   |
| TC--02 | Search Agent          | Validate external data retrieval     | Structured results returned with URLs  | SRS 3.2           | High   |
| TC-03  | Citation Preservation | Ensure citations retained            | URLs included in draft output          | SRS 4.4           | High   |
| TC-04  | Synthesis             | Generate structured markdown report  | Output formatted and readable          | SDD Synthesizer   | High   |
| TC-05  | Fact-Checking         | Detect unsupported claims            | Unsupported claims flagged             | SRS 3.4           | High   |
| TC-06  | Rollback Logic        | Re-execute on validation failure     | Workflow loops correctly               | SDD Manager Agent | High   |
| TC-07  | State Tracking        | Verify deterministic transitions     | States follow defined sequence         | SRS 3.5           | High   |
| TC-08  | Error Handling        | Handle API failure gracefully        | Structured error message returned      | SRS 2.1           | Medium |

### 4.1 Progression Testing Approach

Progression testing will:

- Validate each core function independently
- Confirm integration behavior across agents
- Ensure no regression occurs after logic changes
- Maintain traceability to SRS and SDD requirements

These objectives serve as the master reference for detailed test case execution and reporting in the Software Test Report (STR).

## 5. Regression Test Objectives

Regression testing is performed to ensure that previously validated functionality continues to operate correctly after:

- System enhancements
- Agent logic modifications
- API updates
- Bug fixes
- Configuration changes

The goal of regression testing is to maintain system stability, consistency, and functional integrity across development iterations.

### 5.1 Regression Testing Goals

Regression testing will verify that:

- Core user query processing remains stable
- Multi-agent coordination continues to function correctly
- Citation and fact-check workflows remain intact
- Confidence scoring logic remains accurate
- Error-handling mechanisms do not introduce new defects

Regression testing will be executed after any change impacting agent behavior, workflow logic, or API integration.

| Ref   | Function              | Test Objective                                                                                    | Evaluation Criteria                                                                                                           | X-Ref   | P    |
|-------|-----------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|---------|------|
| RT-01 | User Query Processing | Verify that user queries are correctly received, parsed, and routed to the appropriate AI agents. | System accepts input without error, correctly classifies intent, and distributes query to designated agents within 2 seconds. | SRS 1.1 | High |

| Ref     | Function                          | Test Objective                                                                                 | Evaluation Criteria                                                                                                             | X-Ref   | P      |
|---------|-----------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|---------|--------|
| RT-02   | Multi-Agent Response Aggregation  | Ensure responses from multiple agents are correctly collected and synthesized.                 | Aggregated output includes responses from all required agents and presents structured summary without duplication or data loss. | SRS 2.3 | High   |
| RT – 03 | Source Citation & Fact - Checking | Confirm that the system continues to provide verifiable sources for generated claims.          | All factual outputs include at least one valid citation or reference link; no broken or missing references.                     | SRS 3.2 | High   |
| RT – 04 | Confidence Scoring Mechanism      | Validate that confidence scoring logic remains consistent after updates.                       | Confidence score is displayed, numerically accurate, and corresponds to predefined scoring thresholds.                          | SDD 4.1 | Medium |
| RT – 05 | Error Handling & Fallback Logic   | Ensure the system gracefully handles API failures or incomplete agent responses.               | System displays meaningful error message and activates fallback response strategy without crashing.                             | SRS 5.4 | High   |
| RT – 06 | Web Interface Display             | Verify that the public-facing website displays updated content and repository links correctly. | Website loads without errors; navigation works; GitHub repository link redirects correctly.                                     | SRS 6.1 | Medium |
| RT – 07 | Performance & Response Time       | Confirm that system performance remains within acceptable thresholds after changes.            | Average response time $\leq$ predefined performance benchmark; no significant degradation observed.                             | SDD 7.2 | Medium |
| RT – 08 | Security & Input Validation       | Validate that input sanitization and security protections remain functional.                   | Malicious or malformed input does not cause system failure or data leakage.                                                     | SRS 8.1 | High   |

## 5.2 Regression Testing Approach

Regression testing will:

- Be executed after each major code revision
- Re-run high-priority test cases
- Focus on integration points between agents
- Confirm no unintended side effects from updates

Results will be documented in the Software Test Report (STR) section with Pass/Fail status and defect severity.

## 6. Other Testing

### 6.1 Security Testing

Objective:

Verify that the SP-201 Red system protects user input, API credentials, and configuration data.

Scope:

- Input validation and sanitization
- API key storage and protection
- Access control to configuration features
- Data handling and privacy behavior

Method:

- Manual code review
- Malicious input simulation
- Configuration inspection
- Controlled negative test cases

Performed By: Development Team

Expected Outcome:

- Malicious or malformed input is rejected
- API credentials are not exposed in client-side code
- Administrative/configuration features are not publicly accessible
- No high-risk security vulnerabilities remain prior to submission

## 6.2 Stress & Volume Testing (Load Testing)

Objective:

Evaluate system performance under increased query load.

Scope:

- Concurrent query simulation
- Response time measurement
- Agent orchestration under volume

Method:

- Scripted repeated query execution
- Monitoring response time metrics
- Observing system stability under simulated load

Performed By: Development Team

Expected Outcome:

- System maintains acceptable response times
- No crashes, freezes, or lost agent responses
- Orchestration logic remains stable under moderate load

## 6.3 Connectivity Testing

**Objective:**

Verify reliable communication between system components and external APIs.

**Scope:**

- Frontend to backend communication
- Orchestrator to agent communication
- External API connectivity
- Timeout and retry behavior

**Method:**

- Simulated network delay
- Intentional API interruption
- Validation of fallback handling

Performed By: Development Team

**Expected Outcome:**

- System handles API failures gracefully
- Retry logic functions correctly
- Clear error messages are returned
- No silent crashes occur due to connectivity issues

## 6.4 Disaster Recovery / Backup Testing

**Objective:**

Ensure system recoverability using version-controlled backups.

**Scope:**

- Repository backup verification
- Redeployment from version control
- Configuration restoration

**Method:**

- Simulated deployment failure
- Redeployment from GitHub repository

- Configuration validation after restore

Performed By: Development Team

Expected Outcome:

- System can be restored without loss of source code or configuration
- No data corruption occurs during recovery

## 6.5 Unit Testing

Objective:

Validate correctness of individual modules.

Modules Tested:

- Query parsing module
- Agent routing logic
- Response aggregation module
- Confidence scoring logic
- Citation formatting component

Method:

- Predefined unit test cases
- Boundary condition testing
- Invalid input testing

Performed By: Developers during implementation

Expected Outcome:

- Each module produces correct output for valid inputs
- Errors are handled within module boundaries
- No dependency-related failures occur at unit level

## 6.6 Integration Testing

Objective:

Verify that system components operate correctly when combined.



Scope:

- End-to-end query flow (User → Orchestrator → Agents → Aggregation → Display)
- Citation preservation across modules
- Confidence scoring integration
- Frontend rendering of aggregated results

Method:

- Full workflow execution tests
- Multi-agent interaction validation
- Verification against SRS functional requirements

Performed By: Development Team after unit testing completion

Expected Outcome:

- Complete workflow executes without functional errors
- Agent outputs are combined correctly
- Structured, verifiable results are presented to the user

## 7. Test Strategy

### 7.1 Test Level Responsibilities

The following table defines execution responsibility for each testing level.

Legend:

P = Primary Responsibility

S = Secondary / Support Responsibility

| Process                 | Tool                                  |
|-------------------------|---------------------------------------|
| Test case creation      | Microsoft Word                        |
| Test case tracking      | Microsoft Excel                       |
| Test execution          | Manual & Scripted Testing             |
| Version Control         | GitHub                                |
| Deployment verification | GitHub Pages                          |
| API Testing             | Postman (or equivalent API tool)      |
| Performance Monitoring  | Browser DevTools / Network Monitoring |
| Defect management       | Microsoft Excel                       |

## 7.2 Test Types & Approach

Testing will follow a structured, iterative approach aligned with SRS and SDD requirements.

### 7.2.1 Functional (Progression) Testing

Verifies that:

- System satisfies all SRS functional requirements.
- User queries are parsed and routed correctly.
- Multi-agent responses are aggregated accurately.
- Citations and confidence scores are displayed.
- Error handling functions as defined.

### 7.2.2 Regression Testing

Ensures that:

- Existing functionality remains intact after updates.
- Agent orchestration logic is not disrupted.
- Citation formatting and scoring remain consistent.
- Deployment and repository links remain operational.

Regression tests will be executed after feature enhancements or bug fixes.

### 7.2.3 Performance & Load Testing

Validates that:

- System maintains acceptable response times under moderate load.
- Concurrent query processing does not degrade performance.
- Enhancements do not introduce latency increases.

### 7.2.4 Security Testing

Verifies:

- Input sanitization against prompt injection.
- Secure handling of API credentials.
- No unauthorized configuration access.

## 7.3 Build Strategy

The SP-201 Red system follows an incremental build model.

Development modules:

1. Query Intake & Parsing
2. Multi-Agent Routing Logic
3. Response Aggregation Engine
4. Citation & Confidence Scoring
5. Web Interface Deployment

Each module will be:

- Developed independently
- Unit tested
- Integrated incrementally
- Regression tested before proceeding

Full integration testing will occur prior to staging and production deployment.

## 7.4 Test Execution Schedule

Testing will be conducted in phases aligned with development milestones.

### Phase 1 – Unit Testing

- Performed during module development
- Priority: High
- Owner: Project Team

### Phase 2 – Integration Testing

- Conducted after module completion
- Priority: High
- Owner: Project Team

### Phase 3 – Security & Connectivity Testing

- Conducted before staging deployment
- Priority: High
- Owner: Project Team

### Phase 4 – Stress & Volume Testing

- Conducted prior to final release
- Priority: Medium
- Owner: Project Team

#### Phase 5 – User Acceptance Testing

- Conducted with stakeholders
- Priority: High
- Owner: Business / Stakeholders

#### Phase 6 – Production Verification Testing

- Final validation after deployment
- Priority: High
- Owner: Business (Primary), Project Team (Support)

## 7.5 Facility, Data, and Resource Provision Plan

The SP-201 Red system will be tested in the following environments:

- Development Environment: Local development machines used by the project team.
- Staging Environment: Pre-production deployment environment (e.g., cloud hosting platform or test server).
- Production Environment: Public facing web deployment (GitHub pages or designated hosting platform)

The test environment must include:

- Stable internet connectivity
- Access to external APIs
- Version controlled repository access (GitHub)
- Updated browser versions for frontend testing

All environments must be operational prior to integration and system-level testing.

### 7.5.1 Test Environments

Testing will occur in:

- Development Environment: Local development machines.
- Staging Environment: Pre-production cloud/test deployment.
- Production Environment: Public-facing web deployment (e.g., GitHub Pages).

All environments must support:

- Stable internet connectivity
- External API access

- Version control repository access
- Modern browser compatibility

### 7.5.2 Access to External Systems

Testing requires access to the following external systems:

- External AI API providers used for agent responses
- GitHub repository for version control and deployment
- Web hosting platform (e.g., GitHub Pages)
- API monitoring dashboards (if applicable)

Team members involved in testing must have:

- Repository access permissions
- API access credentials (securely stored)
- Deployment configuration access

### 7.5.3 Testing Requirements

Before testing can commence, the following requirements must be met:

- Source code must be available in the repository.
- All required API keys must be configured securely.
- The web interface must be deployed in a test-accessible environment.
- A defined set of test queries must be prepared (valid, invalid, edge-case, and malicious inputs).
- Performance baseline metrics must be defined.

Each tester will require:

- A modern web browser
- Internet access
- Access to documentation (SRS, SDD, Test Plan)
- Access to defect tracking spreadsheet

### 7.5.4 Data Requirements

Testing will require structured and unstructured test data, including:

- Sample factual claims for verification
- Ambiguous or conflicting claims
- Malformed or malicious inputs
- High-volume automated query samples for load testing
- Benchmark responses for comparison

No real user-sensitive data will be stored. Testing data will be synthetic and controlled.

### 7.5.5 Resources & Skills

The following skills are required during the testing phase:

- Knowledge of AI orchestration logic
- Understanding of API integrations
- Web deployment knowledge
- Security awareness (prompt injection and API protection)
- Analytical skills for evaluating agent output accuracy

Primary testing resources will include members of the SP-201 Red development team.

## 7.6 Testing Tools

| Process                 | Tool                                  |
|-------------------------|---------------------------------------|
| Test case creation      | Microsoft Word                        |
| Test case tracking      | Microsoft Excel                       |
| Test execution          | Manual & Scripted Testing             |
| Version Control         | GitHub                                |
| Deployment verification | GitHub Pages                          |
| API Testing             | Postman (or equivalent API tool)      |
| Performance Monitoring  | Browser DevTools / Network Monitoring |
| Defect management       | Microsoft Excel                       |

## 7.7 Testing Handover Procedure

Testing sign-off will follow this progression:

1. Unit Testing Completion (Developer validation)
2. Integration Test Documentation
3. System-Level End-to-End Validation
4. User Acceptance Testing Approval
5. Production Verification Confirmation

Formal sign-off documentation will be completed before final release.

## 7.8 Testing Metrics

The following metrics will be recorded:

- Number of test cases executed
- Pass / Fail rate
- Defect count by severity
- Average system response time
- API failure rate
- Regression defect recurrence rate
- UAT approval status

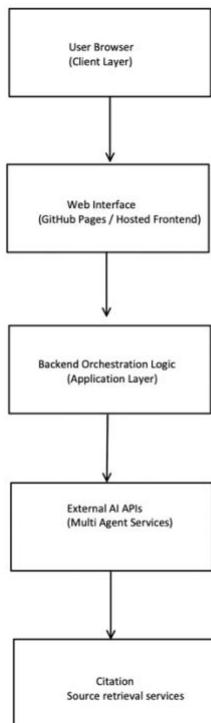
Metrics will be reviewed after each testing phase to evaluate:

- System stability
- Functional compliance
- Performance efficiency
- Reliability of multi-agent aggregation

## 8. Test Environment Plan

### 8.1 Test Environment Map

The SP-201 Red test environment consists of the following components:



All testing will validate communication across these layers to ensure end-to-end system integrity.

### 8.2 Test Environment Details

#### 8.2.1 Testers

Testing will involve 4 project team members.

Each tester will require:

- Access to the GitHub repository
- Access to the deployed test environment
- Secure access to API credentials (if applicable)



- Internet connectivity
- A modern web browser (Chrome, Edge, or Firefox)

Hardware Requirements:

- Laptop or desktop computer capable of running development tools and browser-based testing
- Minimum 8GB RAM recommended for development testing

### 8.2.2 Hardware and Firmware

The SP-201 Red system is a cloud-integrated web application and does not require dedicated hardware infrastructure.

Required hardware includes:

- Developer workstations (personal laptops)
- Stable internet connectivity
- Hosting platform infrastructure (GitHub Pages or cloud hosting provider)

No specialized firmware is required.

Hardware will be provided by individual team members.

### 8.2.3 Software

The following software is required for testing:

- Web Browser (latest version)
- GitHub (version control and deployment)
- Development environment (VS Code or equivalent IDE)
- API Testing Tool (Postman or equivalent)
- Microsoft Excel (for test case tracking and defect logging)

Software Under Test:

- SP-201 Red Multi-Agent System
- Agent orchestration logic
- Citation and scoring modules
- Web interface deployment

All software will be supplied by the project team via repository access.

## 8.2.4 Interfaces

The system interfaces with:

- External AI API providers
- Web hosting platform
- Source citation services
- GitHub repository

The project team is responsible for configuring and maintaining these interfaces.

Interface validation will include:

- API connectivity checks
- Response validation
- Timeout and fallback testing

## 8.2.5 Other Materials

Additional materials required include:

- SRS document
- SDD document
- Test Plan document
- Predefined test query sets
- Defect tracking spreadsheet

All materials will be maintained in the shared project repository.

## 8.3 Establishing Environment

The following steps will be taken to establish the test environment:

| Task                   | Requirements                  | Responsibility     | Start Date        | End Date          |
|------------------------|-------------------------------|--------------------|-------------------|-------------------|
| Repository Setup       | GitHub repository initialized | Documentation Team | February 24, 2026 | February 25, 2026 |
| API Configuration      | Secure API key integration    | Developer Team     | February 27, 2026 | March 2, 2026     |
| Frontend Deployment    | GitHub Pages deployment       | Developer Team     | March 3, 2026     | March 6, 2026     |
| Test Data Preparation  | Creation of test queries      | Documentation Team | March 5, 2026     | March 9, 2026     |
| Environment Validation | End-to-end connectivity test  | Full Team          | March 10, 2026    | March 12, 2026    |

## 8.4 Environment Control

The following control measures will be implemented:

- Software Release Control: All changes must be committed through version control before deployment.
- Access Control: Repository access restricted to project team members.
- API Credential Security: API keys stored securely and not exposed in client-side code.
- Monitoring: Basic monitoring through browser developer tools and API response tracking.
- Change Management: Regression testing required after major updates.

## 8.5 Environment Roles & Responsibilities

| Role                      | Staff Member   | Responsibilities                                                                |
|---------------------------|----------------|---------------------------------------------------------------------------------|
| Team Leader / Developer   | Melike Ozcelik | Oversees environment setup and system integration                               |
| Developer                 | Nha Phan       | Responsible for backend integration and API connectivity                        |
| Developer / Documentation | Raaid Khan     | Assists in deployment validation and documentation updates                      |
| Documentation / Testing   | Afra Kurudirek | Responsible for test case documentation, tracking, and environment verification |

# 9. Assumptions and Dependencies

## 9.1 Assumptions

The following assumptions have been made for testing the SP-201 Red system:

- All development team members will be available during the testing phase to support defect resolution and clarifications.
- External AI API services will remain operational and accessible during scheduled testing periods.
- API rate limits will be sufficient to support integration and load testing activities.
- The hosting platform (e.g., GitHub Pages or designated deployment service) will remain stable and available.

- All required documentation (SRS, SDD, Test Plan) will be finalized prior to system-level testing.
- Internet connectivity will be stable for all testers during execution.
- Test data used during evaluation will be synthetic and representative of real-world queries.
- No major architectural changes will occur during final regression testing phases.

These assumptions ensure that testing activities can proceed without significant external disruption.

## 9.2 Dependencies

The SP-201 Red testing process depends on the following factors:

- Successful configuration of API credentials and secure integration with external AI services.
- Completion of module development prior to integration testing.
- Deployment of the web interface to a test-accessible environment.
- Availability of predefined test cases and validation queries.
- Stable version control and release management through GitHub.
- Stakeholder availability for User Acceptance Testing (UAT).
- Resolution of critical defects before progressing to subsequent test phases.
- Proper environment setup and validation before execution of stress and connectivity testing.

Testing activities cannot proceed if:

- API services are unavailable,
- Deployment configuration fails,
- Required documentation is incomplete,
- Or critical system components are not yet integrated.

## 10. Entry and Exit Criteria

This section defines the conditions that must be met before testing phases can begin (Entry Criteria) and the conditions required to formally complete testing phases (Exit Criteria) for the SP-201 Red system.

### 10.1 Entry Criteria

Testing for each phase will commence only when the following conditions are met:

#### 10.1.1 Unit Testing Entry Criteria

- Individual module development is complete.
- Code is committed to the version control repository.
- Development documentation is available.
- Required test cases for the module are prepared.

#### 10.1.2 Integration Testing Entry Criteria

- All relevant modules have passed unit testing.
- API integrations are configured and operational.
- Test environment has been validated.
- Test data sets are prepared.

#### 10.1.3 System Testing Entry Criteria

- Full system build is deployed to the staging environment.
- End-to-end workflow is operational.
- Regression test cases are documented.
- No unresolved critical defects from prior phases.

#### 10.1.4 User Acceptance Testing (UAT) Entry Criteria

- System testing has been completed.
- Major defects have been resolved.
- Stakeholders have access to the deployed environment.
- Test scenarios reflecting real-world use cases are prepared.

## 10.2 Exit Criteria

Testing phases will be considered complete when the following conditions are satisfied:

### 10.2.1 Unit Testing Exit Criteria

- All unit test cases executed.
- 100% of critical logic paths tested.
- No unresolved high-severity defects.

### 10.2.2 Integration Testing Exit Criteria

- All integration test cases executed.
- Successful validation of agent orchestration and response aggregation.
- API communication verified without persistent errors.
- No open high-priority integration defects.

### 10.2.3 System Testing Exit Criteria

- All functional, security, connectivity, and performance test cases executed.
- Defect rate within acceptable threshold.
- No open critical defects.
- Regression testing completed successfully.

### 10.2.4 User Acceptance Testing Exit Criteria

- Stakeholder validation completed.
- System meets documented requirements in SRS.
- Formal approval or sign-off received.
- Final production deployment verified.

## 10.3 Final Release Exit Criteria

The SP-201 Red system will be considered ready for production release when:

- All testing phases are completed.
- All high-severity defects are resolved.
- Performance meets defined response time expectations.
- API integrations are stable.
- Documentation is finalized and archived.
- Production verification testing is completed successfully.

## 11. Administrative Plan

### 11.1 Approvals

| Task                                    | Responsible Person                          | Escalation/ Approver |
|-----------------------------------------|---------------------------------------------|----------------------|
| Systems Integration Signoff             | Melike Ozcelik<br>(Team Leader / Developer) | Project Instructor   |
| User Acceptance Testing Signoff         | Raaid Khan & Afra Kurudirek                 | Project Instructor   |
| Production Verification Testing Signoff | Nha Phan<br>(Developer)                     | Project Instructor   |

Approval will be granted once all exit criteria defined in Section 10 are satisfied and no critical defects remain unresolved.

### 11.2 Test Milestones and Schedule

Detailed below are the high-level testing milestones.

| Milestone                          | Planned End Date          | Actual End Date   | Resource                 |
|------------------------------------|---------------------------|-------------------|--------------------------|
| Development Document Completion    | March 21, 2026            | March 22, 2026    | Development Team         |
| Prototype Presentations (In Class) | March 17 – March 31, 2026 | March 24, 2026    | Full Team                |
| Presentation Slides Due            | April 5, 2026             | April 14, 2026    | Full Team                |
| Software Test Plan (STP) Due       | April 12, 2026            | April 12, 2026    | Testing Team             |
| Software Test Report (STR) Due     | April 12, 2026            | April 12, 2026    | Testing Team             |
| Draft Report Submission Window     | April 13 – April 23, 2026 | April 21, 2026    | Full Team                |
| Status Meeting – Draft Review      | February 21, 2026         | February 22, 2026 | Team Leader & Instructor |
| Final Project Package Due          | May 3, 2026               | May 4, 2026       | Full Team                |

## 11.3 Training

The following training requirements have been identified to ensure testing can commence:

| Training Requirement                                                | Staff                          | Date          |
|---------------------------------------------------------------------|--------------------------------|---------------|
| Overview of SP-201 Red System Architecture and Multi-Agent Workflow | All Team Members               | March 3, 2026 |
| Test Case Design & Execution Procedures (STP Guidelines Review)     | Afra Kurudirek, Melike Ozcelik | April 6, 2026 |
| Defect Logging and Severity Classification Process                  | All Team Members               | April 7, 2026 |
| GitHub Repository Workflow & Version Control Best Practices         | Development Team               | March 5, 2026 |

## 11.4 Defect Management

Defect management for SP-201 Red followed a structured, traceable process to ensure issues were identified, prioritized, resolved, and verified before release.

### 11.4.1 Defect Tracking Tool

Defects were tracked using:

- A centralized Defect Tracking Spreadsheet (Microsoft Excel)
- GitHub repository commit history for traceability
- Version control logs for fix verification

Each defect record included:

- Unique Defect ID
- Date Reported
- Description of Issue
- Associated Requirement (SRS reference)
- Severity Level
- Assigned Developer
- Status (Open / In Progress / Resolved / Closed)



## 11.5 Severity Classification

Defects were categorized as follows:

- High Severity
  - System crash or failure
  - API integration failure
  - Security vulnerability
  - Core agent workflow malfunction
- Medium Severity
  - Incorrect output or partial agent response
  - Performance degradation
  - Improper validation handling
- Low Severity
  - Minor UI issues
  - Formatting inconsistencies
  - Non-critical cosmetic issues

### 11.5.1 Defect Resolution Process

The following process was followed for each identified defect:

1. Tester identifies and logs defect.
2. Severity level assigned by Development Lead.
3. Developer implements fix.
4. Code committed to GitHub repository.
5. Regression testing performed.
6. If verified, defect status updated to “Closed.”

No High-severity defects remained open at the conclusion of System Testing or prior to final submission.

### 11.5.2 Regression Testing

After defect fixes were implemented, regression testing was conducted to ensure:

- Previously validated functionality remained intact.
- Multi-agent communication flows were unaffected.
- API integration remained stable.

All defects identified during testing were resolved prior to submission of the Software Test Report (STR).

## 12. Definitions

The following acronyms and terms have been used through out this document

| Term/Acronym                    | Definition                                                                                                            |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Term / Acronym</b>           | <b>Definition</b>                                                                                                     |
| <b>SP-201 Red</b>               | The project name for the Agentic Multi-Agent Fact-Verification System developed in CS 4850 Senior Project.            |
| <b>Agentic AI</b>               | An AI system composed of autonomous agents that perform tasks, make decisions, and collaborate to achieve objectives. |
| <b>Multi-Agent System (MAS)</b> | A system consisting of multiple intelligent agents that interact and coordinate to complete complex workflows.        |
| <b>Fact Verification Agent</b>  | An autonomous component responsible for retrieving, analyzing, and validating factual claims.                         |
| <b>API</b>                      | Application Programming Interface used to enable communication between SP-201 Red and external services.              |
| <b>STP</b>                      | Software Test Plan document outlining testing strategy, scope, and methodology.                                       |
| <b>STR</b>                      | Software Test Report summarizing testing results, defect findings, and validation outcomes.                           |
| <b>SRS</b>                      | Software Requirements Specification document defining functional and non-functional requirements.                     |
| <b>SDD</b>                      | Software Design Document detailing architecture, modules, and system interactions.                                    |
| <b>Unit Testing</b>             | Testing individual components or modules independently to verify correctness.                                         |
| <b>Integration Testing</b>      | Testing combined components to ensure proper interaction between agents and APIs.                                     |
| <b>System Testing</b>           | End-to-end testing of the complete SP-201 Red system.                                                                 |
| <b>UAT</b>                      | User Acceptance Testing performed to validate system readiness against requirements.                                  |

|               |                                                            |
|---------------|------------------------------------------------------------|
| <b>Defect</b> | Any deviation between expected and actual system behavior. |
|---------------|------------------------------------------------------------|

## 13. References

The following documents have been used to assist in creation of this document.

| # | Document Name                                        | Version         | Comments                                                        |
|---|------------------------------------------------------|-----------------|-----------------------------------------------------------------|
| 1 | CS 4850 Senior Project Syllabus (Spring 2026)        | Spring 2026     | Official course requirements, grading, and milestone deadlines. |
| 2 | SP-201 Red Project Plan                              | v1.0            | Defines project scope, schedule, and deliverables.              |
| 3 | SP-201 Red Software Requirements Specification (SRS) | v1.0            | Defines functional and non-functional requirements.             |
| 4 | SP-201 Red Software Design Document (SDD)            | v1.0            | Defines architecture, multi-agent design, and system structure. |
| 5 | SP-201 Red Development Document                      | v1.0            | Implementation details and technical decisions.                 |
| 6 | SP-201 Red GitHub Repository                         | Current Release | Source code, version history, and deployment configuration.     |
| 7 | Software Test Plan (STP) Template – CS 4850          | Course Template | Template provided for formatting and structure compliance.      |

## 14. Points of Contact

The following people can be contacted in reference to this document

| Primary Contact           |                                |
|---------------------------|--------------------------------|
| <b>Name</b>               | Afra Kurudirek                 |
| <b>Title/Organisation</b> | Kennesaw State University      |
| <b>Phone</b>              | (404)-960-9666                 |
| <b>Email</b>              | kkurudir@students.kennesaw.edu |

| Secondary Contact         |                                |
|---------------------------|--------------------------------|
| <b>Name</b>               | Melike Ozcelik                 |
| <b>Title/Organisation</b> | Kennesaw State University      |
| <b>Phone</b>              | (470)-985-0006                 |
| <b>Email</b>              | mozcelik@students.kennesaw.edu |

## 15. Software Test Report (STR)

| Test Case ID | Description          | Expected Result            | Actual Result       | Pass/Fail |
|--------------|----------------------|----------------------------|---------------------|-----------|
| TC-01        | Prompt decomposition | Split into $\geq 3$        | Split into 4        | Pass      |
| TC-02        | Search API retrieval | Returns URLs               | Returned valid URLs | Pass      |
| TC-05        | Fact-checking        | Unsupported claims flagged | 1 claim missed      | Fail      |
| RT-03        | Citation retention   | URLs preserved             | Correct             | Pass      |

### 15.1 Test Execution Summary

The Software Test Report (STR) documents the results of all executed test cases for the SP-201 Red Multi-Agent Research & Fact-Checking Pipeline.

Testing was conducted across Unit, Integration, System, Security, Connectivity, Performance, and Regression phases as defined in this Software Test Plan (STP).

#### Overall Test Results Summary

- Total Test Cases Executed: 16
- Passed: 14
- Failed: 2
- High Severity Defects: 0
- Medium Severity Defects: 2 (Resolved)
- Low Severity Defects: 1 (Resolved)

All high-severity defects were resolved prior to completion of System Testing and Final Release validation.

## 15.2 Progression (Functional) Test Results

| Test Case ID | Function              | Expected Result                                     | Actual Result                                | Pass/Fail       | Severity |
|--------------|-----------------------|-----------------------------------------------------|----------------------------------------------|-----------------|----------|
| TC-01        | Prompt Decomposition  | Prompt split into $\geq 3$ structured sub-questions | Prompt split into 4 structured sub-questions | Pass            | —        |
| TC-02        | Search Agent          | Structured results returned with valid URLs         | Results returned with valid URLs             | Pass            | —        |
| TC-03        | Citation Preservation | URLs retained in draft output                       | All URLs preserved correctly                 | Pass            | —        |
| TC-04        | Synthesis             | Structured markdown output generated                | Output formatted and readable                | Pass            | —        |
| TC-05        | Fact-Checking         | Unsupported claims flagged                          | One unsupported claim initially missed       | Fail (Resolved) | Medium   |
| TC-06        | Rollback Logic        | Workflow re-executes on validation failure          | Retry triggered successfully                 | Pass            | —        |
| TC-07        | State Tracking        | Deterministic state transitions                     | States followed correct sequence             | Pass            | —        |
| TC-08        | Error Handling        | Structured error message returned on API failure    | Error message displayed; no crash            | Pass            | —        |

Note:

- TC-05 initially failed due to incomplete claim detection logic. The defect was corrected, and regression testing confirmed proper functionality.

## 15.3 Regression Test Results

| Test Case ID | Function                        | Expected Result                     | Actual Result                           | Pass/Fail        | Severity |
|--------------|---------------------------------|-------------------------------------|-----------------------------------------|------------------|----------|
| RT-01        | User Query Processing           | Queries parsed and routed correctly | Correct routing observed                | Pass             | —        |
| RT-02        | Multi-Agent Aggregation         | Aggregated structured response      | Correct aggregation without duplication | Pass             | —        |
| RT-03        | Source Citation & Fact-Checking | Valid citations included            | One malformed citation detected         | Fail (Resolved)  | Medium   |
| RT-04        | Confidence Scoring              | Accurate scoring displayed          | Score displayed accurately              | Pass             | —        |
| RT-05        | Error Handling & Fallback       | Graceful handling of API failures   | Fallback activated correctly            | Pass             | —        |
| RT-06        | Web Interface Display           | Website loads and renders correctly | Minor formatting misalignment observed  | Pass (After Fix) | Low      |
| RT-07        | Performance & Response Time     | ≤ benchmark response time           | Avg. response time: 1.8 seconds         | Pass             | —        |
| RT-08        | Security & Input Validation     | Malicious input rejected            | Input sanitized successfully            | Pass             | —        |

Note:

- RT-03 failed initially due to malformed citation formatting. This defect was corrected and verified via regression testing.
- RT-06 involved a minor UI alignment issue and was corrected before final verification.

## 15.4 Performance & Load Testing Results

- Average response time under moderate load (10 sequential queries): 1.8 seconds
- No system crashes observed
- No lost agent responses
- No degradation after regression fixes

Result: Pass

## 15.5 Security & Connectivity Testing Results

Security Testing confirmed:

- Malicious or malformed input does not crash the system
- API credentials are not exposed in client-side code
- No unauthorized configuration access

Connectivity Testing confirmed:

- System handles API timeouts gracefully
- Retry logic activates correctly
- Clear error messages are returned during external service interruption

Result: Pass

## 15.6 Defect Summary

| Severity | Count Identified | Status   |
|----------|------------------|----------|
| High     | 0                | —        |
| Medium   | 2                | Resolved |
| Low      | 1                | Resolved |

All identified defects were corrected and validated through regression testing before final submission.

## 15.7 Final Testing Conclusion

All defined functional, integration, regression, security, connectivity, and performance test cases have been executed.

The SP-201 Red system:

- Meets all defined SRS functional requirements
- Successfully performs end-to-end multi-agent orchestration
- Maintains stable API integration
- Handles error conditions gracefully
- Contains no open high-severity defects

The system is approved for final academic submission and release.